

# Algorithms & Data Structures

## Fall 2024

### Exercise Sheets

#### *Weeks 1-2: C++ Basics, Complexity, and ADTs*

Submit your solutions via feedback fruits in Brightspace, Week 1-2. Your handin should consist of a **zip file containing a pdf and any code** you were required to produce, along with the main files and dependencies to allow the instructor to compile and test your code. In the pdf please include **screenshots of the most important parts of the code**.

#### Exercises

- (1) Number of integers in a vector
  - (a) (Week 1) Write a program that generates random  $M$  integers and puts them in an `vector`. Then it generates another  $N$  random integers and counts how many of them are in the array using an iterator.
  - (b) (Week 2) Estimate the worst-case complexity of your program and justify with a line-by-line explanation of the number of steps taken in the program.

- (2) (Old exam question, week 2) What is the time complexity (Big-O) of myMethod? Argue for your answer:

```
int myMethod(int N)
{
    int x = 0;
    int y = 0;
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            for (int k = 0; k < N * sqrt(N); k++)
            {
                x++;
            }
        }
    }
    for (int i = 0; i < N * N; i++)
    {
        y++;
    }
    return x + y;
}
```

- (3) (Week 2) For each of the following four program fragments:
- (a) Give an analysis of the running time in Big-Oh notation.
  - (b) Implement the code inside a C function, and give the running time for several values of  $N$ .
  - (c) Compare your analysis with the actual running times.
  - (d) Does compiling/running programs with the `-O` optimization flag change anything in your analysis?

|                                                                                                                                      |                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <pre>(1) sum = 0;     for (i = 0; i &lt; n; ++i) {         ++sum;     }</pre>                                                        | <pre>(2) sum = 0;     for (i = 0; i &lt; n; ++i) {         for (j = 0; j &lt; n; ++j) {             ++sum;         }     }</pre> |
| <pre>(3) sum = 0;     for (i = 0; i &lt; n; ++i) {         for (j = 0; j &lt; n * n; ++j) {             ++sum;         }     }</pre> | <pre>(4) sum = 0;     for (i = 0; i &lt; n; ++i) {         for (j = 0; j &lt; i; ++j) {             ++sum;         }     }</pre> |

- (4) (a) (Old exam question, week 1) Write an implementation of the **Abstract Data Type** MaxHeap described below. The implementation **must** be done by using the standard class vector from C++ **and it does not have to be efficient**. If you need to iterate through the vector, you **must** use an iterator and its methods and operators begin(), end(), ++ and \*

```
class MaxHeap
{
public:
    // is the heap empty?
    virtual bool isEmpty() const = 0;

    // number of elements in the heap
    virtual int size() = 0;

    // add an element to the heap
    virtual void insert(const int x) = 0;

    // find the maximum element in the heap
    virtual const int findMax() const = 0;

    // delete and return the maximum element of the heap
    virtual int deleteMax() = 0;
};
```

- (b) (Week 2) What is the time complexity of the five operations of MaxHeap? You must argue for your answers

- (5) (a) Create and implement a C++ class called *Library* that manages books. Each book (which should be a class as well) has an identifier (an integer provided when creating the book) and a category (an integer between 0 and 15 inclusive, provided when constructing the book). The library should allow adding new books, borrowing books, returning borrowed books, and displaying books available as well as borrowed. There should be a function that returns the number of books in a given category. When adding a new book, a warning (to `std::cout`) should be issued if the number of books in any **valid category** exceeds twice the average number of books per **valid category**, to ensure the user is balanced. A **valid category** is category for which books have been added to. Create a main function that tests all features of the library. The code below is provided as a **starting point that you can modify**.

```
int main() {
    Library myLibrary;
    // Adding books to the library
    myLibrary.addBook(1, 3); // Book ID: 1, Category: 3
    myLibrary.addBook(2, 7); // Book ID: 2, Category: 7
    myLibrary.addBook(3, 3); // Book ID: 3, Category: 3
    myLibrary.addBook(4, 15); // Book ID: 4, Category: 15
    myLibrary.addBook(5, 3); // Book ID: 5, Category: 3 (Warning should trigger here)
    myLibrary.addBook(6, 0); // Book ID: 6, Category: 0
    // Display all books in the library
    std::cout << "\nAll books in the library:\n";
    myLibrary.displayAllBooks();
    // Display available books
    std::cout << "\nAvailable books in the library:\n";
    myLibrary.displayAvailableBooks();
    // Borrow a book by ID
    std::cout << "\nBorrowing Book ID 2:\n";
    if (myLibrary.borrowBook(2)) {
        std::cout << "Book ID 2 borrowed successfully.\n";
    } else {
        std::cout << "Failed to borrow Book ID 2.\n";
    }
    // Attempt to borrow the same book again
    std::cout << "\nAttempting to borrow Book ID 2 again:\n";
    if (myLibrary.borrowBook(2)) {
        std::cout << "Book ID 2 borrowed successfully.\n";
    } else {
        std::cout << "Failed to borrow Book ID 2.\n";
    }
    // Display borrowed books
    std::cout << "\nBorrowed books in the library:\n";
    myLibrary.displayBorrowedBooks();
    // Return a borrowed book by ID
    std::cout << "\nReturning Book ID 2:\n";
    if (myLibrary.returnBook(2)) {
        std::cout << "Book ID 2 returned successfully.\n";
    } else {
        std::cout << "Failed to return Book ID 2.\n";
    }
    // Display available books after returning
    std::cout << "\nAvailable books in the library after returning Book ID 2:\n";
    myLibrary.displayAvailableBooks();
    // Display the number of books in a specific category
    int categoryToCheck = 3;
    std::cout << "\nNumber of books in category " << categoryToCheck << ":\n";
    << myLibrary.countBooksInCategory(categoryToCheck) << std::endl;

    return 0;
}
```

- (b) Justify the choice of the data structures you chose to maintain the books in the library.  
(c) Provide the complexity of each operation in the library, and justify.